ACCESO A DATOS - 2.º DAM

UP 1 – XML: introducción y lectura con SAX y DOM

1. ¿Qué es XML?

XML (eXtensible Markup Language) es un formato de texto que permite guardar e intercambiar información de forma estructurada mediante etiquetas. A diferencia de un fichero binario, podemos abrirlo con un editor y leer su contenido.

Por ejemplo, una aplicación puede utilizar XML para guardar configuraciones o intercambiar datos con otra aplicación. Es otro formato de persistencia, igual que los ficheros de texto, CSV o JSON que ya hemos trabajado.

> **IDEA CLAVE**
> Serializar consiste en transformar un objeto en una secuencia de bytes para guardarlo en un fichero. Deserializar es leer esos bytes y reconstruir el objeto.

2. Estructura de un documento XML

Este fichero almacena información de dos videojuegos:

```
<?xml version="1.0" encoding="UTF-8"?>
<videojuegos>
  <videojuego id="1">
    <titulo>Tetris</titulo>
    <plataforma>PC</plataforma>
    <precio>19.99</precio>
  </videojuego>
  <videojuego id="2">
    <titulo>Mario Kart</titulo>
    <plataforma>Switch</plataforma>
    <precio>49.99</precio>
  </videojuego>
</videojuegos>
```

Parte	Qué significa
Declaración XML	Indica la versión y la codificación del documento; es opcional.
Elemento raíz	<videojuegos> contiene todos los demás elementos.
Elementos	<videojuego>, <titulo>, <precio>… organizan los datos.
Atributos	id="1" añade información a la etiqueta de un elemento.
Contenido	Tetris, PC y 19.99 son valores guardados dentro de las etiquetas.

3. ¿Cuándo está bien formado un XML?

Un XML está bien formado cuando respeta las reglas de escritura del lenguaje:

- Cada etiqueta abierta debe cerrarse (o utilizar una etiqueta vacía).
- Las etiquetas deben estar bien anidadas: no pueden cruzarse.
- Debe existir un único elemento raíz.
- Las mayúsculas importan: <Titulo> y <titulo> son diferentes.
- Los valores de los atributos se escriben entre comillas.

> **COMPRUÉBALO**
> ¿Qué pasaría si eliminamos </titulo> del ejemplo? El documento dejaría de estar bien formado y un analizador XML normal produciría un error al leerlo.

4. DOM y SAX: ¿cómo trabaja Java con XML?

Java puede leer XML de distintas formas. Las dos que vamos a conocer son DOM y SAX. DOM carga todo el documento como un árbol de elementos; SAX lo recorre de principio a fin y va avisando al programa cuando encuentra etiquetas o texto. Solo practicaremos la lectura con DOM. Existe también StAX, otra alternativa de lectura por eventos, que no estudiaremos ahora.

Tecnología	Idea básica
DOM	Carga el XML completo en memoria como un árbol de nodos. Permite buscar elementos y consultar sus datos; lo utilizaremos en el ejemplo.
SAX	Procesa el XML secuencialmente y genera eventos al encontrar etiquetas o texto. No necesita cargar todo el documento en memoria.
StAX	Otra opción para leer XML mediante eventos. Solo la nombramos.

> **IDEA CLAVE**
> Con DOM podemos buscar elementos por su nombre y consultar su contenido. SAX también permite leer XML, pero funciona por eventos; StAX es otra alternativa que solo mencionamos

5. Un ejemplo de lectura con DOM

El siguiente código abre un XML de la carpeta del proyecto y muestra el contenido de todas sus etiquetas <titulo>. Observa las tres acciones: cargar, buscar y leer.

```
import java.io.File;
import javax.xml.parsers.DocumentBuilderFactory;
import org.w3c.dom.Document;
import org.w3c.dom.NodeList;

public class LeerXML {
    public static void main(String[] args) {
        try {
            Document doc = DocumentBuilderFactory.newInstance()
                    .newDocumentBuilder().parse(new File("videojuegos.xml"));
            NodeList titulos = doc.getElementsByTagName("titulo");
            for (int i = 0; i < titulos.getLength(); i++) {
                System.out.println(titulos.item(i).getTextContent());
            }
        } catch (Exception e) {
            System.out.println("Error al leer el XML: " + e.getMessage());
        }
    }
}
```

Document representa el documento cargado; getElementsByTagName("titulo") localiza los elementos; getTextContent() obtiene el texto almacenado. En este ejemplo se utiliza un fichero local preparado por nosotros.

6. Un ejemplo de lectura con SAX

SAX lee el documento de principio a fin. En lugar de cargarlo entero en memoria, ejecuta métodos cuando encuentra una etiqueta o su contenido. Vamos a leer un fichero libros.xml y mostrar el título y el autor de cada libro.

Contenido de libros.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<biblioteca>
    <libro>
        <titulo>El principito</titulo>
        <autor>Antoine de Saint-Exupéry</autor>
    </libro>
    <libro>
        <titulo>Don Quijote</titulo>
        <autor>Miguel de Cervantes</autor>
    </libro>
</biblioteca>
```

Código Java: LeerLibrosSAX.java

```
import java.io.File;
import javax.xml.parsers.SAXParser;
import javax.xml.parsers.SAXParserFactory;
import org.xml.sax.Attributes;
import org.xml.sax.helpers.DefaultHandler;

public class LeerLibrosSAX {
    public static void main(String[] args) {
        try {
            SAXParserFactory fabrica = SAXParserFactory.newInstance();
            SAXParser analizador = fabrica.newSAXParser();
            analizador.parse(new File("libros.xml"), new ManejadorLibros());
        } catch (Exception e) {
            System.out.println("Error al leer XML: " + e.getMessage());
        }
    }
}

class ManejadorLibros extends DefaultHandler {
    private StringBuilder contenido = new StringBuilder();

    @Override
    public void startElement(String uri, String localName,
                             String qName, Attributes atributos) {
        contenido.setLength(0);
    }

    @Override
    public void characters(char[] caracteres, int inicio, int longitud) {
        contenido.append(caracteres, inicio, longitud);
    }

    @Override
    public void endElement(String uri, String localName, String qName) {
        if (qName.equals("titulo")) {
            System.out.println("Título: " + contenido.toString().trim());
        } else if (qName.equals("autor")) {
            System.out.println("Autor: " + contenido.toString().trim());
        } else if (qName.equals("libro")) {
            System.out.println("--------------------");
        }
        contenido.setLength(0);
    }
}
```

¿Qué hace cada método?

- startElement(): se ejecuta al encontrar una etiqueta de apertura.
- characters(): recibe fragmentos de texto. Por eso los acumulamos hasta que se cierre la etiqueta.
- endElement(): se ejecuta al cerrar una etiqueta; aquí mostramos el título o el autor.

Resultado esperado: Título: El principito · Autor: Antoine de Saint-Exupéry; después, Título: Don Quijote · Autor: Miguel de Cervantes.

> **RECUERDA**
> DOM representa todo el XML como un árbol en memoria y SAX lo procesa secuencialmente mediante eventos.

7. Resumen

Concepto	Explicación
XML	Formato de texto estructurado mediante etiquetas.
Raíz	Elemento único que engloba el resto del documento.
Atributo	Dato incluido en una etiqueta de apertura.
Bien formado	Etiquetas cerradas y anidadas correctamente, entre otras reglas.
DOM	Carga el XML completo en memoria como un árbol de nodos. Permite buscar elementos y consultar sus datos; lo utilizaremos en el ejemplo.
SAX	Lee secuencialmente mediante eventos; no lo programaremos en esta unidad.